

Assignment and scheduling (2/2)

7.5–7.7

Outline

Classes with setup

Completion bonus

Total tardiness

Summary

Classes with setup

Introduction: Classes with setup

- ▶ A single machine, and the jobs are divided into **classes**: using a class costs a setup, in euros and in minutes
- ▶ It is a knapsack with fixed costs per group
- ▶ Here the activation link is born already in the **disaggregated** form, $x_j \leq y_c$, derived from the CNF of a Boolean implication

Problem 7.5: classes with setup (1/2)

- ▶ A company has $n \in \mathbb{Z}_{\geq 1}$ jobs that can be executed on a machine with an available processing time of $a \in \mathbb{Q}_{>0}$ minutes
- ▶ For each job j , $t_j \in \mathbb{Q}_{>0}$ is the processing time and $r_j \in \mathbb{Q}_{>0}$ the revenue in euros if the job is executed
- ▶ The jobs are partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes: for each class c , the set $\mathcal{J}_c \subseteq \{1, \dots, n\}$ contains the jobs of the class

Problem 7.5: classes with setup (2/2)

- ▶ If the machine executes jobs of a class c , a setup cost of $f_c \in \mathbb{Q}_{\geq 0}$ euros is incurred and a setup time of $s_c \in \mathbb{Q}_{\geq 0}$ minutes is needed
- ▶ The machine cannot execute jobs in parallel
- ▶ The company wants to choose the jobs to execute **maximising the profit**

The problem in words

- ▶ **We decide** which jobs to execute and, consequently, which classes to activate
- ▶ **The objective:** revenues minus setup costs
- ▶ **The constraint:** times of the jobs plus setup times of the activated classes within the availability
- ▶ A knapsack with fixed costs per group: the activation link, this time **disaggregated** from the start

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, \dots, n\}$
a	$\in \mathbb{Q}_{>0}$	availability of the machine
t_j, r_j	$\in \mathbb{Q}_{>0}$	time and revenue of job j
q	$\in \mathbb{Z}_{\geq 2}$	number of classes, $c \in \{1, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, \dots, n\}$	jobs of class c (partition)
f_c, s_c	$\in \mathbb{Q}_{\geq 0}$	setup cost and setup time of class c

Decision variables

We introduce the following $n + q$ binary variables:

$$\begin{cases} x_j = 1 \text{ if job } j \text{ is executed, } 0 \text{ otherwise,} \\ y_c = 1 \text{ if at least one job of class } c \text{ is executed,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall c \in \{1, 2, \dots, q\}.$$

Definition 1: Model 7.5: classes with setup

$$\begin{aligned} \max \quad & \sum_{j=1}^n r_j x_j - \sum_{c=1}^q f_c y_c \\ \text{s.t.} \quad & \sum_{j=1}^n t_j x_j + \sum_{c=1}^q s_c y_c \leq a, \quad (1) \\ & x_j - y_c \leq 0, \quad \forall c, \forall j \in \mathcal{J}_c \quad (2) \\ & x_j \in \{0, 1\}, \quad y_c \in \{0, 1\}, \quad \forall j, \forall c \end{aligned}$$

Description of the objective function and constraints

- ▶ The objective function maximises the profit, the difference between the revenue of the executed jobs and the setup cost of the activated classes
- ▶ Constraint (1) ensures that the time of the executed jobs, plus the setup times of the activated classes, does not exceed the availability (1 constraint)
- ▶ Constraints (2) link the execution variables with the activation ones: if a job of a class is executed, the class is activated (n constraints, one per job, because the classes partition the jobs)

The link between the variables (1/3)

- ▶ For each class c , if at least one job of the class is executed, the class is activated:

$$\underbrace{(x_j \text{ OR } x_s \text{ OR } \dots)}_{j,s,\dots \in \mathcal{J}_c} \implies y_c$$

- ▶ By contraposition (De Morgan)

$$\text{NOT } y_c \implies (\text{NOT } x_j \text{ AND NOT } x_s \text{ AND } \dots) :$$

if the class is not activated, none of its jobs is executed

The link between the variables (2/3)

- ▶ The implication is equivalent to the Boolean expression $\text{NOT}(x_j \text{ OR } x_s \text{ OR } \dots) \text{ OR } y_c$, which by De Morgan and distributivity becomes the CNF

$$(\text{NOT } x_j \text{ OR } y_c) \text{ AND } (\text{NOT } x_s \text{ OR } y_c) \text{ AND } \dots$$

- ▶ It translates into the $|\mathcal{J}_c|$ linear constraints $1 - x_j + y_c \geq 1$, that is

$$x_j - y_c \leq 0, \quad \forall j \in \mathcal{J}_c$$

exactly constraints (2)

- ▶ Check: if $x_j = 1$ then $y_c = 1$; if $y_c = 0$ then $x_j = 0$ for every $j \in \mathcal{J}_c$

The link between the variables (3/3)

- ▶ Conversely, an activated class has at least one executed job:

$$y_c \implies (x_j \text{ OR } x_s \text{ OR } \dots)$$

- ▶ It is not imposed by the constraints, but it follows *without loss of optimality*: setting $y_c = 0$ stays feasible, frees s_c minutes and *does not decrease* the objective, because $f_c \geq 0$
- ▶ Since f_c may be zero, the improvement is not necessarily strict: the correct conclusion is “*there always exists* an optimum in which classes with no jobs are not activated”, not “in every optimum”
- ▶ The sign of the data decides the strength of the conclusion

The model in gurobipy

```
# classes are lists of indices: J =  
    [[0,1],[2,3],[4,5,6]]  
m = gp.Model("classes_setup")  
x = m.addVars(n, vtype=GRB.BINARY, name="x")  
y = m.addVars(q, vtype=GRB.BINARY, name="y")  
m.setObjective(gp.quicksum(r[j] * x[j] for j in range(  
    n))  
    - gp.quicksum(f[c] * y[c] for c in range(q)),  
    GRB.MAXIMIZE)  
m.addConstr(gp.quicksum(t[j] * x[j] for j in range(n))  
    + gp.quicksum(s[c] * y[c] for c in range(q)) <= a,  
    name="availability")  
m.addConstrs((x[j] - y[c] <= 0 for c in range(q)  
    for j in J[c]), name="link")
```

The instance

$n = 7$ jobs, $q = 3$ classes $\mathcal{J}_1 = \{1, 2\}$, $\mathcal{J}_2 = \{3, 4\}$, $\mathcal{J}_3 = \{5, 6, 7\}$,
availability $a = 50$:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$	$j = 7$
r_j	10	6	8	6	7	9	5
t_j	5	10	8	6	9	5	6

	$c = 1$	$c = 2$	$c = 3$
f_c	10	5	4
s_c	10	12	6

The model for the instance

$$\begin{aligned}
 &\max 10x_1 + 6x_2 + 8x_3 + 6x_4 + 7x_5 + 9x_6 + 5x_7 - 10y_1 - 5y_2 - 4y_3 \\
 &\text{s.t. } 5x_1 + 10x_2 + 8x_3 + 6x_4 + 9x_5 + 5x_6 + 6x_7 + 10y_1 + 12y_2 + 6y_3 \leq 50 \\
 &\quad x_1 \qquad \qquad \qquad -y_1 \qquad \qquad \leq 0 \\
 &\quad \quad x_2 \qquad \qquad \qquad -y_1 \qquad \qquad \leq 0 \\
 &\quad \quad \quad x_3 \qquad \qquad \qquad -y_2 \qquad \leq 0 \\
 &\quad \quad \quad \quad x_4 \qquad \qquad \qquad -y_2 \qquad \leq 0 \\
 &\quad \quad \quad \quad \quad x_5 \qquad \qquad \qquad -y_3 \leq 0 \\
 &\quad \quad \quad \quad \quad \quad x_6 \qquad \qquad \qquad -y_3 \leq 0 \\
 &\quad \quad \quad \quad \quad \quad \quad x_7 \qquad \qquad \qquad -y_3 \leq 0 \\
 &\quad x_1, \quad x_2, \quad x_3, \quad x_4, \quad x_5, \quad x_6, \quad x_7 \qquad \qquad \qquad \in \{0, 1\} \\
 &\quad \qquad \qquad y_1, \quad y_2, \quad y_3 \in \{0, 1\}
 \end{aligned}$$

Constructive heuristic: the primal bound

- ▶ Class by class, job by job
- ▶ The first job of a class is executed if its time and the setup of the class fit
- ▶ The following ones are executed if their time fits
- ▶ It is a constructive heuristic: a single solution, one element at a time, never backtracking

Run on the instance (1/2)

- ▶ **Step 1.** Class 1 not active: $s_1 + t_1 = 10 + 5 = 15 \leq 50$;
 $y[1] = 1$, $x[1] = 1$, $ra = 35$
- ▶ **Step 2.** Class 1 active: $t_2 = 10 \leq 35$; $x[2] = 1$, $ra = 25$
- ▶ **Step 3.** Class 2 not active: $s_2 + t_3 = 12 + 8 = 20 \leq 25$;
 $y[2] = 1$, $x[3] = 1$, $ra = 5$
- ▶ **Step 4.** Class 2 active: $t_4 = 6 > 5$; job 4 is skipped
- ▶ **Steps 5–7.** Class 3 not active: $s_3 + t_j \in \{15, 11, 12\}$, all > 5 ;
jobs 5, 6, 7 are skipped

Run on the instance (2/2)

Solution

$$\bar{x} = (1, 1, 1, 0, 0, 0, 0), \quad \bar{y} = (1, 1, 0),$$

profit

$$10 + 6 + 8 - 10 - 5 = 9, \quad z(MILP) \geq LB = 9.$$

- The scanning order of the classes matters a lot: here class 1 consumes $10 + 10$ minutes between setup and first job

LP relaxation and dual

- ▶ Relaxing $x_j \geq 0$ and $y_c \geq 0$, with duals $\pi \geq 0$ for constraint (1) and $\lambda_j \geq 0$ for constraints (2)

Definition 2: Dual of the LP relaxation of model 7.5

$$\min \quad a \pi$$

$$s.t. \quad t_j \pi + \lambda_j \geq r_j, \quad \forall j \quad (3)$$

$$s_c \pi - \sum_{j \in \mathcal{J}_c} \lambda_j \geq -f_c, \quad \forall c \quad (4)$$

$$\pi \geq 0, \quad \lambda_j \geq 0, \quad \forall j$$

Description of the dual

- ▶ π is the price of one unit of time, λ_j the price of the link between job j and the setup of its class
- ▶ The objective is the value of all the available time a
- ▶ Constraints (3) are the columns of the x_j : the time t_j consumed by job j , priced at π , plus the price of its link, must cover the revenue r_j
- ▶ Constraints (4) are the columns of the y_c : the setup of class c consumes s_c of time and releases the links of all the jobs in the class; the balance cannot fall below $-f_c$

The dual of the instance

$$\begin{array}{llll} \min & 50\pi & & \\ \text{s.t.} & 5\pi + \lambda_1 & & \geq 10 \\ & 10\pi & + \lambda_2 & \geq 6 \\ & 8\pi & & + \lambda_3 \geq 8 \\ & 6\pi & & + \lambda_4 \geq 6 \\ & 9\pi & & + \lambda_5 \geq 7 \\ & 5\pi & & + \lambda_6 \geq 9 \\ & 6\pi & & + \lambda_7 \geq 5 \\ & 10\pi - \lambda_1 - \lambda_2 & & \geq -10 \\ & 12\pi & - \lambda_3 - \lambda_4 & \geq -5 \\ & 6\pi & & - \lambda_5 - \lambda_6 - \lambda_7 \geq -4 \\ & \pi, \lambda_1, \lambda_2, \lambda_3, \lambda_4, \lambda_5, \lambda_6, \lambda_7 & & \geq 0 \end{array}$$

A hand-built dual solution

- ▶ With $\bar{\lambda}_j = 0$ constraints (3) ask $\pi \geq r_j/t_j$ for every job, and constraints (4) are satisfied for every $\pi \geq 0$:

$$\bar{\pi} = \max_j \frac{r_j}{t_j} = \max\left\{\frac{10}{5}, \frac{6}{10}, \frac{8}{8}, \frac{6}{6}, \frac{7}{9}, \frac{9}{5}, \frac{5}{6}\right\} = 2$$

- ▶ Value $50 \cdot 2 = 100$, hence $LB = 9 \leq z(MILP) \leq UB = 100$
- ▶ The reading: π is the value of one minute of machine; if every minute were worth as much as the most profitable job per minute, the whole machine would be worth 100 euros
- ▶ A coarse bound, as “knapsack” bounds often are: it ignores setups and costs

What the solver says (1/2)

- ▶ $z(LP) = 425/13 = 32.7$ (optimum of the dual, reached with $\pi = \frac{17}{26}$ and some $\lambda_j > 0$: the solver does not settle for $\lambda = 0$)
- ▶ With the upper bounds, $z(LP^+) = 329/13 = 25.3$
- ▶ The integer optimum is $z(MILP) = 21$: classes 2 and 3 activated, jobs 3, 4, 5, 6 executed ($8 + 6 + 9 + 5 + 12 + 6 = 46 \leq 50$), profit $30 - 9$
- ▶ The heuristic stays at 9: a 57% gap, the worst of the family

What the solver says (2/2)

LB (heuristic)	UB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$
9	100	425/13	329/13	21

- ▶ The scanning order matters: a constructive heuristic that orders the classes by potential profit would do much better

Additional considerations

- ▶ $y_c \leq 1$ (q) are valid inequalities that strengthen the relaxation; $x_j \leq 1$ (n) are implied by $y_c \leq 1$ and constraints (2)
- ▶ The direction “no job, hence class not activated” is the clause NOT y_c OR x_j OR x_s OR $\dots$, that is

$$\sum_{j \in \mathcal{J}_c} x_j \geq y_c, \quad \forall c \in \{1, 2, \dots, q\}$$

not valid, but it can be added without losing the optimum (for $f_c \geq 0$ and $s_c \geq 0$)

- ▶ The **aggregated** form $\sum_{j \in \mathcal{J}_c} x_j \leq |\mathcal{J}_c| y_c$ has the same set of integer solutions and a weaker relaxation: here the model is born already in the right form

Exercise: a class subordinate to another

- ▶ The jobs of class 3 use a tool that is mounted during the setup of class 1: class 3 can be activated only if class 1 is activated too
- ▶ Write the constraint
- ▶ Find the new optimum for the instance

The final comparison (1/2)

The three numbers that every problem of the course produces:

	value	what it is
LB	9	heuristic solution
UB	100	certified bound
$z(MILP)$	21	MILP optimum

The final comparison (2/2)

- ▶ Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 57.1\%$$

- ▶ Certified gap: in absolute terms $UB - LB = 91$; as a percentage, against the heuristic solution,

$$\frac{UB - LB}{|LB|} = 1011.1\%$$

- ▶ Two very distant bounds: both can be improved

A worked variant: a single class (1/2)

- ▶ Changing class requires a cleaning of the machine the company wants to avoid altogether: at most one class can be activated
- ▶ A *set packing* constraint on the activations:

$$\sum_{c=1}^q y_c \leq 1$$

(one linear constraint)

- ▶ At most one y_c equals 1, and with it, through constraints (2), only the jobs of that class can be executed

A worked variant: a single class (2/2)

- ▶ On the instance the optimum drops to 17: the best class alone is class 3, with all its jobs ($9 + 5 + 6 + 6 = 26 \leq 50$), profit $21 - 4 = 17$

	value	what it is
UB	$\frac{157}{5}$	hand-built dual certificate
LB	17	heuristic solution
$z(LP)$	17	relaxation without the bounds
$z(MILP)$	17	MILP optimum

The bound improves a great deal over the recipe of the base problem.

Summary: Classes with setup

- ▶ Execution variables x_j and class activation variables y_c
- ▶ $x_j \leq y_c$ for every $j \in \mathcal{J}_c$: the **disaggregated** form, derived from the CNF of the implication
- ▶ The opposite direction follows *without loss of optimality*: with $f_c \geq 0$ one concludes “there exists an optimum”, not “in every optimum”
- ▶ Bounds: $LB = 9$ (heuristic), $UB = 100$ (hand dual), $z(MILP) = 21$: the widest gap of the family

Completion bonus

Introduction: Completion bonus

- ▶ Two “**if and only if**” conditions: a bonus collected if and only if a class is complete, and a reduction of the availability if and only if classes are mixed
- ▶ One rewards, the other penalises
- ▶ We will see that for each “if and only if” it suffices to impose *one direction*: the other is imposed by the objective (or by the capacity constraint)

Problem 7.6: completion bonus (1/2)

- ▶ A company has $n \in \mathbb{Z}_{\geq 1}$ jobs that can be executed on a machine with an available processing time of $a \in \mathbb{Q}_{>0}$ minutes
- ▶ For each job j , $t_j \in \mathbb{Q}_{>0}$ is the processing time and $r_j \in \mathbb{Q}_{>0}$ the revenue if executed
- ▶ The jobs are partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes $\mathcal{J}_1, \dots, \mathcal{J}_q$

Problem 7.6: completion bonus (2/2)

- ▶ The availability is reduced by $u \in \mathbb{Q}_{>0}$ minutes *if and only if* the machine executes jobs of at least two different classes
- ▶ For each class c , an extra revenue of $v_c \in \mathbb{Q}_{>0}$ euros is obtained *if and only if* all the jobs of the class are executed
- ▶ The company wants to choose the jobs to execute **maximising the total revenue**

The problem in words

- ▶ **We decide** which jobs to execute
- ▶ **The objective:** revenues of the jobs plus bonuses of the complete classes
- ▶ **The constraint:** times of the jobs, plus the reduction u if the classes involved are at least two, within the availability
- ▶ Two “if and only if”s: one rewards (completion), one penalises (mixing)

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, \dots, n\}$
a	$\in \mathbb{Q}_{>0}$	availability of the machine
t_j, r_j	$\in \mathbb{Q}_{>0}$	time and revenue of job j
q	$\in \mathbb{Z}_{\geq 2}$	number of classes, $c \in \{1, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, \dots, n\}$	jobs of class c (partition)
v_c	$\in \mathbb{Q}_{>0}$	bonus if class c is complete
u	$\in \mathbb{Q}_{>0}$	reduction if jobs of at least two classes are executed

Decision variables

We introduce the following $n + q + 1$ binary variables:

$$\begin{cases} x_j = 1 \text{ if job } j \text{ is executed,} \\ y_c = 1 \text{ if all the jobs of class } c \text{ are executed,} \\ z = 1 \text{ if jobs of at least two different classes are executed,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall c \in \{1, 2, \dots, q\}.$$

Definition 3: Model 7.6: completion bonus

$$\max \quad \sum_{j=1}^n r_j x_j + \sum_{c=1}^q v_c y_c$$

$$s.t. \quad x_j - y_c \geq 0, \quad \forall c, \forall j \in \mathcal{J}_c \quad (1)$$

$$x_j + x_i - z \leq 1, \quad c < g, j \in \mathcal{J}_c, i \in \mathcal{J}_g \quad (2)$$

$$\sum_{j=1}^n t_j x_j + u z \leq a, \quad (3)$$

$$x_j, y_c, z \in \{0, 1\}$$

Description of the objective function and constraints

- ▶ The objective function maximises the total revenue: revenues of the executed jobs plus bonuses of the complete classes
- ▶ Constraints (1) link execution and completion: if a class is declared complete, all its jobs are executed (n constraints)
- ▶ Constraints (2) link execution and mixing: if two jobs of different classes are executed, then $z = 1$ (one per mixed pair)
- ▶ Constraint (3) ensures that the time of the jobs, plus the reduction u when classes are mixed, does not exceed the availability

The link between the variables: the four implications

Four implications: two for y_c , two for z .

- ▶ In each pair one is imposed by the constraint, the other by the optimum
- ▶ And this is how an “**if and only if**” is modelled: the constraint imposes the direction that could be violated in favour of the objective, the objective does the rest

The link for y_c (1/2)

- ▶ If not all the jobs of the class are executed, the class is not complete; by contraposition

$$y_c \implies (x_j \text{ AND } x_i \text{ AND } \dots), \quad j, i, \dots \in \mathcal{J}_c$$

- ▶ The Boolean expression $(x_j \text{ AND } x_i \text{ AND } \dots) \text{ OR NOT } y_c$ becomes, by distributivity, the CNF

$$(x_j \text{ OR NOT } y_c) \text{ AND } (x_i \text{ OR NOT } y_c) \text{ AND } \dots$$

that is $x_j + (1 - y_c) \geq 1$: these are constraints (1), $x_j - y_c \geq 0$

The link for y_c (2/2)

- ▶ Check: if $y_c = 1$, every constraint $x_j \geq y_c$ forces $x_j = 1$; if some $x_j = 0$ with $j \in \mathcal{J}_c$, the constraint forces $y_c = 0$
- ▶ The opposite direction, “if all the jobs are executed then the class is complete”, is *not* imposed by the constraints
- ▶ It follows from the objective in every optimum: since $v_c > 0$, if all the x_j of the class are 1 and $y_c = 0$, setting $y_c = 1$ stays feasible and increases the objective by v_c
- ▶ Therefore in every optimum the bonus is collected as soon as the class is complete

The link for z (1/2)

- ▶ For every pair of jobs j, i of different classes:

$$x_j \text{ AND } x_i \implies z,$$

by contraposition $\text{NOT } z \implies \text{NOT } (x_j \text{ AND } x_i)$

- ▶ By De Morgan, $\text{NOT } x_j \text{ OR } \text{NOT } x_i \text{ OR } z$, that is
 $(1 - x_j) + (1 - x_i) + z \geq 1$: these are constraints (2),
 $x_j + x_i - z \leq 1$
- ▶ Check: if $x_j = x_i = 1$ the constraint forces $z = 1$; if $z = 0$ it becomes $x_j + x_i \leq 1$

The link for z (2/2)

- ▶ Conversely, if the executed jobs all lie in one class, $z = 0$
- ▶ It is not imposed by the constraints, and follows from the *capacity constraint* without loss of optimality: setting $z = 0$ stays feasible (constraints (2) become $x_j + x_i \leq 1$, true because no mixed pair is executed), frees $u > 0$ minutes and does not change the objective, where z does not appear
- ▶ Therefore there always exists an optimum in which $z = 1$ only if classes are mixed

The model in gurobipy

```
pairs = [(j, i) for c in range(q) for g in range(c+1,
           q)
           for j in J[c] for i in J[g]]
m = gp.Model("classes_bonus")
x = m.addVars(n, vtype=GRB.BINARY, name="x")
y = m.addVars(q, vtype=GRB.BINARY, name="y")
z = m.addVar(vtype=GRB.BINARY, name="z")
m.setObjective(gp.quicksum(r[j] * x[j] for j in range(
    n))
    + gp.quicksum(v[c] * y[c] for c in range(q)), GRB.
    MAXIMIZE)
m.addConstrs((x[j] - y[c] >= 0 for c in range(q)
    for j in J[c]), name="all")
m.addConstrs((x[j] + x[i] - z <= 1 for (j, i) in pairs
    ),
    name="mixed")
```

The instance

$n = 6$ jobs, $q = 3$ classes $\mathcal{J}_1 = \{1, 2\}$, $\mathcal{J}_2 = \{3, 4\}$, $\mathcal{J}_3 = \{5, 6\}$,
 $a = 50$, $u = 10$:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
r_j	10	5	20	12	10	22
t_j	5	15	25	15	10	38

	$c = 1$	$c = 2$	$c = 3$
v_c	5	4	10

Twelve mixed pairs, hence twelve constraints (2).

The model for the instance (1/3)

$$\max 10x_1 + 5x_2 + 20x_3 + 12x_4 + 10x_5 + 22x_6 + 5y_1 + 4y_2 + 10y_3$$

$$\text{s.t. } x_1 \qquad \qquad \qquad -y_1 \qquad \qquad \geq 0$$

$$\qquad x_2 \qquad \qquad \qquad -y_1 \qquad \qquad \geq 0$$

$$\qquad \qquad x_3 \qquad \qquad \qquad -y_2 \qquad \qquad \geq 0$$

$$\qquad \qquad \qquad x_4 \qquad \qquad \qquad -y_2 \qquad \qquad \geq 0$$

$$\qquad \qquad \qquad \qquad x_5 \qquad \qquad -y_3 \geq 0$$

$$\qquad \qquad \qquad \qquad \qquad x_6 \qquad \qquad -y_3 \geq 0$$

The model for the instance (2/3)

$$x_1 + x_3 - z \leq 1$$

$$x_1 + x_4 - z \leq 1$$

$$x_2 + x_3 - z \leq 1$$

$$x_2 + x_4 - z \leq 1$$

$$x_1 + x_5 - z \leq 1$$

$$x_1 + x_6 - z \leq 1$$

$$x_2 + x_5 - z \leq 1$$

$$x_2 + x_6 - z \leq 1$$

The model for the instance (3/3)

$$\begin{array}{rcll} x_3 & +x_5 & -z & \leq 1 \\ x_3 & & +x_6 & -z \leq 1 \\ & x_4 & +x_5 & -z \leq 1 \\ & x_4 & & +x_6 -z \leq 1 \\ 5x_1+15x_2+25x_3+15x_4+10x_5+38x_6 & & +10z & \leq 50 \\ x_1, & x_2, & x_3, & x_4, & x_5, & x_6 & & \in \{0,1\} \\ & & & & & & y_1,y_2,y_3 & z \in \{0,1\} \end{array}$$

Constructive heuristic: the primal bound

- ▶ Class by class
- ▶ From the second class on, the first executed job also pays the reduction u
- ▶ At the end of a class, if all its jobs are executed, the bonus is collected

Run on the instance (1/2)

- ▶ **Steps 1–2.** Class 1: $t_1 = 5 \leq 50$, $x[1] = 1$, $ra = 45$;
 $t_2 = 15 \leq 45$, $x[2] = 1$, $ra = 30$. Class complete: $y[1] = 1$
- ▶ **Step 3.** Class 2, reduction not applied yet: $t_3 + u = 35 > 30$,
job 3 is skipped
- ▶ **Step 4.** $t_4 + u = 25 \leq 30$: $x[4] = 1$, $z = 1$, $ra = 5$. Class 2
incomplete
- ▶ **Steps 5–6.** Class 3: $t_5 = 10 > 5$ and $t_6 = 38 > 5$, skipped

Run on the instance (2/2)

Solution

$$\bar{x} = (1, 1, 0, 1, 0, 0), \quad \bar{y} = (1, 0, 0), \quad \bar{z} = 1,$$

revenue

$$10 + 5 + 12 + 5 = 32, \quad z(MILP) \geq LB = 32.$$

LP relaxation and dual

- Duals $\pi_j \leq 0$ for constraints (1), $\lambda_{ji} \geq 0$ for constraints (2), $\mu \geq 0$ for constraint (3)

Definition 4: Dual of the LP relaxation of model 7.6

$$\min \sum_{c < g} \sum_{j \in \mathcal{J}_c} \sum_{i \in \mathcal{J}_g} \lambda_{ji} + a \mu$$

$$s.t. \quad \pi_j + \sum_{i \notin \mathcal{J}_c} \lambda_{ji} + t_j \mu \geq r_j, \quad \forall c, \forall j \in \mathcal{J}_c \quad (4)$$

$$- \sum_{j \in \mathcal{J}_c} \pi_j \geq v_c, \quad \forall c \quad (5)$$

$$- \sum_{c < g} \sum_{j \in \mathcal{J}_c} \sum_{i \in \mathcal{J}_g} \lambda_{ji} + u \mu \geq 0, \quad (6)$$

with $\pi_j \leq 0$, $\lambda_{ji} \geq 0$, $\mu \geq 0$.

Description of the dual (1/2)

- ▶ π_j is the (non-positive) price of the link “if the class is rewarded, then job j must be done”
- ▶ λ_{ji} is the price of the constraint on the mixed pair (j, i)
- ▶ μ is the price of one unit of time
- ▶ The objective adds up the prices of all the mixed pairs and the value of the available time a

Description of the dual (2/2)

- ▶ Constraints (4) are the columns of the x_j : doing job j releases the link with its class, touches all the mixed-pair constraints that contain it and consumes t_j of time; the total must cover the revenue r_j
- ▶ Constraints (5) are the columns of the y_c : the bonus v_c must be covered by the links of the jobs that make up the class
- ▶ Constraint (6) is the column of z : the variable that flags two different classes releases all the mixed-pair constraints and consumes u of time

A hand-built dual solution (1/2)

- Constraints (5) ask the π_j of every class to “cover” the bonus: it suffices to load it on a single job per class,

$$\bar{\pi}_1 = -5, \quad \bar{\pi}_3 = -4, \quad \bar{\pi}_5 = -10,$$

the others 0

- With $\bar{\lambda} = 0$ constraint (6) is satisfied for every $\mu \geq 0$ and constraints (4) ask $\mu \geq (r_j - \bar{\pi}_j)/t_j$

A hand-built dual solution (2/2)

$$\bar{\mu} = \max\left\{\frac{10+5}{5}, \frac{5}{15}, \frac{20+4}{25}, \frac{12}{15}, \frac{10+10}{10}, \frac{22}{38}\right\} = 3,$$

value $a\bar{\mu} = 150$, hence $LB = 32 \leq z(MILP) \leq UB = 150$.

- ▶ The bound is dominated by job 1, short and with the bonus loaded on it
- ▶ Spreading the bonus over the two jobs of the class ($\pi_1 = \pi_2 = -2.5$) would lower it: a good exercise

What the solver says

- ▶ $z(LP) = z(LP^+) = 5280/113 = 46.7$
- ▶ The integer optimum is $z(MILP) = 42$: class 3 alone, complete, jobs 5 and 6 ($10 + 38 = 48 \leq 50$, $z = 0$), revenue $10 + 22 + 10$
- ▶ The heuristic, started from class 1, stops at 32: a 24% gap

LB (heuristic)	UB (hand dual)	$z(LP)$	$z(MILP)$
32	150	5280/113	42

Additional considerations (1/2)

- ▶ The optimality direction for y_c can be imposed explicitly with the clause $\text{NOT } x_j \text{ OR } \text{NOT } x_i \text{ OR } \dots \text{ OR } y_c$, that is

$$\sum_{j \in \mathcal{J}_c} x_j - y_c \leq |\mathcal{J}_c| - 1, \quad \forall c$$

(q optimality-preserving constraints, not valid)

- ▶ The optimality direction for z : for every class c ,

$$z \leq \sum_{j \notin \mathcal{J}_c} x_j$$

(q constraints). These too preserve the optimum without being valid

Additional considerations (2/2)

- ▶ Constraints (2) are as many as the mixed pairs: 12 in the instance, in general $\sum_{c < g} |\mathcal{J}_c| |\mathcal{J}_g|$
- ▶ An **aggregated** form with class variables,

$$w_c \geq x_j \quad (j \in \mathcal{J}_c), \quad \sum_{c=1}^q w_c - 1 \leq (q-1)z,$$

uses q more variables and $n+1$ constraints instead of the pairs

- ▶ Same set of integer solutions, different relaxation: comparing them is a good exercise

Exercise: penalty for a class started and not finished

- ▶ Starting a class without completing it costs $w = 3$ euros of reorganisation
- ▶ Model the penalty: which new variable is needed?
- ▶ In which direction must its link be proved?

The final comparison (1/2)

The three numbers that every problem of the course produces:

	value	what it is
LB	32	heuristic solution
UB	150	certified bound
$z(MILP)$	42	MILP optimum

The final comparison (2/2)

- ▶ Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 23.8\%$$

- ▶ Certified gap: in absolute terms $UB - LB = 118$; as a percentage, against the heuristic solution,

$$\frac{UB - LB}{|LB|} = 368.8\%$$

A worked variant: at least one job per class (1/2)

- ▶ A union agreement requires executing at least one job of every class
- ▶ A covering constraint per class:

$$\sum_{j \in \mathcal{J}_c} x_j \geq 1, \quad \forall c \in \{1, 2, \dots, q\}$$

- ▶ With $q \geq 2$ classes all touched, for every pair of classes there is a mixed pair of executed jobs, and constraints (2) force $z = 1$
- ▶ The reduction u is certain and can be substituted in the capacity constraint ($a - u$ instead of a , eliminating z)

A worked variant: at least one job per class (2/2)

- ▶ On the instance the optimum drops from 42 to 40: for instance jobs 1, 3 and 5 ($5 + 25 + 10 + 10 = 50$), revenue 40, no complete class

	value	what it is
UB	107	hand-built dual certificate
LB	32	heuristic solution
$z(LP)$	$\frac{309}{7}$	relaxation without the bounds
$z(MILP)$	40	MILP optimum

Summary: Completion bonus

- ▶ Three families of variables: execution x_j , completion y_c , mixing z
- ▶ For every “if and only if” it suffices to impose **one direction** with the constraints: the other is imposed by the objective (bonus, $v_c > 0$) or by the capacity constraint (reduction, $u > 0$)
- ▶ The mixed-pair constraints are many: an aggregated form with class variables exists
- ▶ Bounds: $LB = 32$ (heuristic), $UB = 150$ (hand dual), $z(LP) = 5280/113$, $z(MILP) = 42$

Total tardiness

Introduction: Total tardiness

- ▶ A single machine and a **sequence** of jobs to decide, with due dates to respect
- ▶ The order is a **disjunction**: “either j before i or i before j ”
- ▶ It is linearised with a binary precedence variable and a **big-M** constant: we will see why $M = \sum_j t_j$ is enough, and why the relaxation of these models is extremely weak

Problem 7.7: total tardiness

- ▶ A company needs to process $n \in \mathbb{Z}_{\geq 1}$ jobs on a single machine
- ▶ For each job j , $t_j \in \mathbb{Q}_{>0}$ is the processing time in minutes and $d_j \in \mathbb{Q}_{>0}$ the due date in minutes
- ▶ The tardiness of job j is $\tau_j = \max\{0, \kappa_j - d_j\}$, where κ_j is the completion time
- ▶ The machine processes one job at a time; the company wants a sequence that **minimises the total tardiness**

The problem in words

- ▶ **We decide** the order of the jobs
- ▶ **The objective:** the sum of the tardiness values
- ▶ **The constraints:** for every pair, one of the two precedes the other; whoever comes later finishes at least t minutes after the completion of whoever comes first
- ▶ The order is a **disjunction**: it is linearised with a binary variable and a big-M

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, \dots, n\}$
t_j	$\in \mathbb{Q}_{>0}$	processing time of job j
d_j	$\in \mathbb{Q}_{>0}$	due date of job j
M	$\in \mathbb{Q}_{>0}$	big-M constant, $M = \sum_{j=1}^n t_j$

Decision variables

We introduce $n(n - 1)$ binary precedence variables and $2n$ non-negative variables:

$$\begin{cases} s_{ji} = 1 \text{ if job } j \text{ precedes job } i, 0 \text{ otherwise,} \\ \kappa_j = \text{completion time of job } j, \\ \tau_j = \text{tardiness of job } j, \end{cases}$$

$$\forall j, i \in \{1, 2, \dots, n\}, j \neq i.$$

Definition 5: Model 7.7: total tardiness

$$\min \sum_{j=1}^n \tau_j$$

$$\text{s.t. } s_{ji} + s_{ij} = 1, \quad \forall j < i \quad (1)$$

$$-M s_{ji} - \kappa_j + \kappa_i \geq t_i - M, \quad \forall j \neq i \quad (2)$$

$$-\kappa_j + \tau_j \geq -d_j, \quad \forall j \quad (3)$$

$$\kappa_j \geq t_j, \quad \forall j \quad (4)$$

$$s_{ji} \in \{0, 1\}, \quad \kappa_j \geq 0, \quad \tau_j \geq 0$$

Description of the objective function and constraints

- ▶ The objective function minimises the total tardiness
- ▶ Constraints (1) link the precedence variables of each pair: either j precedes i or i precedes j ($n(n-1)/2$ constraints)
- ▶ Constraints (2) link precedences and completion times through the **big-M**: if j precedes i , job i cannot complete before j is completed and i has been processed ($n(n-1)$ constraints)
- ▶ Constraints (3), together with $\tau_j \geq 0$, define the tardiness; constraints (4) impose that every job completes no earlier than its processing time

The link between the variables: the precedence (1/2)

- For every pair j, i :

$$s_{ji} = 1 \implies \kappa_i \geq \kappa_j + t_i$$

and by contraposition $\kappa_i < \kappa_j + t_i \implies s_{ji} = 0$

- It is imposed with the big-M: $\kappa_i \geq \kappa_j + t_i - M(1 - s_{ji})$, that is constraints (2)
- If $s_{ji} = 1$ the constraint becomes $\kappa_i \geq \kappa_j + t_i$; if $s_{ji} = 0$ it becomes $\kappa_i \geq \kappa_j + t_i - M$, i.e. it is “switched off”

The link between the variables: the precedence (2/2)

- ▶ Why is $M = \sum_{j=1}^n t_j$ enough? There always exists an optimal solution with no idle time: if the machine stops, the following jobs are moved earlier and no completion, hence no tardiness, gets worse
- ▶ On such a solution every $\kappa_j \leq \sum_j t_j = M$, hence

$$\kappa_j + t_i - M \leq t_i \leq \kappa_i$$

- ▶ It is the pattern “big-M: conditional constraints and disjunctions”: the big-M *switches off* the constraint

The link between the variables: the tardiness

- ▶ For every job j , $\tau_j \geq \max\{0, \kappa_j - d_j\}$ is imposed directly by constraints (3) and by $\tau_j \geq 0$: there is no implication to prove, the link *is* the inequality
- ▶ There is instead an optimality implication: if the job finishes in time, the tardiness is zero,

$$\kappa_j \leq d_j \implies \tau_j = 0$$

- ▶ The constraints do not impose it, but it follows from the objective: if $\kappa_j \leq d_j$ and $\tau_j > 0$, setting $\tau_j = 0$ stays feasible and decreases the objective by $\tau_j > 0$
- ▶ **Synthesis:** in every optimum $\tau_j = \max\{0, \kappa_j - d_j\}$, exactly the tardiness

The model in gurobipy

```
M = sum(t)
m = gp.Model("tardiness")
s = m.addVars([(j, i) for j in range(n) for i in range
    (n)
    if j != i], vtype=GRB.BINARY, name="s")
kappa = m.addVars(n, name="kappa")
tau = m.addVars(n, name="tau")
m.setObjective(tau.sum(), GRB.MINIMIZE)
m.addConstrs((s[j, i] + s[i, j] == 1 for j in range(n)
    for i in range(j + 1, n)), name="order")
m.addConstrs((-M * s[j, i] - kappa[j] + kappa[i] >= t[
    i] - M
    for j in range(n) for i in range(n) if j != i),
    name="precedence")
```

The instance

$n = 3$ jobs, $M = 5 + 4 + 6 = 15$:

	$j = 1$	$j = 2$	$j = 3$
t_j	5	4	6
d_j	3	4	10

- ▶ Three order constraints, six precedence constraints, three tardiness constraints and three minimum-time constraints

The model for the instance (1/2)

$$\begin{array}{llllllll}
 \min & & & & & & & \tau_1 + \tau_2 + \tau_3 \\
 \text{s.t.} & s_{12} & & +s_{21} & & & & = 1 \\
 & & s_{13} & & +s_{31} & & & = 1 \\
 & & & s_{23} & & +s_{32} & & = 1 \\
 & -15s_{12} & & & & -\kappa_1 + \kappa_2 & & \geq -11 \\
 & & -15s_{13} & & & -\kappa_1 & +\kappa_3 & \geq -9 \\
 & & & -15s_{21} & & +\kappa_1 - \kappa_2 & & \geq -10 \\
 & & & & -15s_{23} & & -\kappa_2 + \kappa_3 & \geq -9 \\
 & & & & & -15s_{31} & +\kappa_1 & -\kappa_3 \geq -10 \\
 & & & & & & -15s_{32} & +\kappa_2 - \kappa_3 \geq -11
 \end{array}$$

The model for the instance (2/2)

$$\begin{array}{rcll} -\kappa_1 & +\tau_1 & \geq & -3 \\ & -\kappa_2 & +\tau_2 & \geq -4 \\ & & -\kappa_3 & +\tau_3 \geq -10 \\ \kappa_1 & & & \geq 5 \\ & \kappa_2 & & \geq 4 \\ & & \kappa_3 & \geq 6 \\ s_{12}, s_{13}, s_{21}, s_{23}, s_{31}, s_{32} & & \in & \{0, 1\} \\ \kappa_1, \kappa_2, \kappa_3, \tau_1, \tau_2, \tau_3 & \geq & 0 & \end{array}$$

Constructive heuristic: the primal bound

- ▶ The simplest: process the jobs in the given order $1 \rightarrow 2 \rightarrow 3$ (FIFO)
- ▶ A better rule orders by increasing due date (EDD, *earliest due date*), which here coincides
- ▶ The sequence is built one job at a time, never backtracking

Run on the instance

- ▶ **Step 1.** Job 1: $\kappa_1 = 5$, $\tau_1 = \max\{0, 5 - 3\} = 2$
- ▶ **Step 2.** Job 2: $\kappa_2 = 5 + 4 = 9$, $\tau_2 = \max\{0, 9 - 4\} = 5$
- ▶ **Step 3.** Job 3: $\kappa_3 = 9 + 6 = 15$, $\tau_3 = \max\{0, 15 - 10\} = 5$

Solution $\bar{s}_{12} = \bar{s}_{13} = \bar{s}_{23} = 1$, $\bar{\kappa} = (5, 9, 15)$, $\bar{\tau} = (2, 5, 5)$, value 12:

$$z(MILP) \leq UB = 12.$$

LP relaxation and dual

- ▶ Relaxing $s_{ji} \geq 0$, with duals α_{ji} free for (1), $\beta_{ji} \geq 0$ for (2), $\gamma_j \geq 0$ for (3) and $\delta_j \geq 0$ for (4)

Definition 6: Dual of the LP relaxation of model 7.7

$$\begin{aligned} \max \quad & \sum_{j < i} \alpha_{ji} + \sum_{j \neq i} (t_i - M) \beta_{ji} - \sum_j d_j \gamma_j + \sum_j t_j \delta_j \\ \text{s.t.} \quad & \alpha_{ji} - M \beta_{ji} \leq 0, \quad \alpha_{ji} - M \beta_{ij} \leq 0, \quad \forall j < i \\ & - \sum_{i \neq j} \beta_{ji} + \sum_{i \neq j} \beta_{ij} - \gamma_j + \delta_j \leq 0, \quad \forall j \\ & \gamma_j \leq 1, \quad \forall j \end{aligned}$$

with $\alpha_{ji} \gtrless 0$ and $\beta_{ji}, \gamma_j, \delta_j \geq 0$.

Description of the dual (1/2)

- ▶ α_{ji} is the price of the ordering choice between jobs j and i
- ▶ β_{ji} the price of the precedence written with the big-M
- ▶ γ_j the price of the constraint that defines the tardiness of job j
- ▶ δ_j that of its minimum completion time
- ▶ The objective collects these prices with the right-hand sides of the corresponding constraints

Description of the dual (2/2)

- ▶ The first two groups of constraints are the columns of the s_{ji} and of the s_{ij} : the ordering choice enters the equality with coefficient 1 and the precedence with $-M$; the balance cannot be positive
- ▶ The third group is the column of the κ_j : the completion time enters the precedences with opposite signs, the tardiness with -1 and the minimum time with $+1$
- ▶ The last group is the column of the τ_j : tardiness costs 1 in the primal objective, so its price cannot exceed 1

The dual of the instance

The objective of the dual for the instance:

$$\begin{aligned} \max \quad & \alpha_{12} + \alpha_{13} + \alpha_{23} \\ & - 11\beta_{12} - 9\beta_{13} - 10\beta_{21} - 9\beta_{23} - 10\beta_{31} - 11\beta_{32} \\ & - 3\gamma_1 - 4\gamma_2 - 10\gamma_3 + 5\delta_1 + 4\delta_2 + 6\delta_3 \end{aligned}$$

- The constraints are $\alpha_{ji} \leq 15\beta_{ji}$ and $\alpha_{ji} \leq 15\beta_{ij}$ for every pair, the three columns of the κ_j and the three limits $\gamma_j \leq 1$

A hand-built dual solution

- ▶ Every β_{ji} has coefficient $t_i - M < 0$: it pays to keep them at zero, and then the first constraints impose $\alpha = 0$
- ▶ What is left is γ and δ , with $\delta_j \leq \gamma_j \leq 1$: every job contributes at most

$$t_j \delta_j - d_j \gamma_j \leq t_j - d_j,$$

positive only if the job is late even when processed first

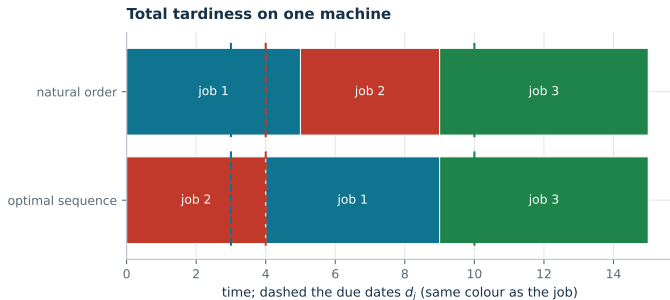
- ▶ Only job 1 is ($t_1 = 5 > d_1 = 3$): $\bar{\gamma}_1 = \bar{\delta}_1 = 1$, all the rest 0, value $-3 + 5 = 2$
- ▶ Hence $LB = 2 \leq z(MILP) \leq UB = 12$: job 1 will be at least 2 minutes late wherever it is placed

What the solver says (1/2)

- ▶ $z(LP) = 2$: the hand-built solution is optimal for the dual
- ▶ The LP relaxation of this model is **extremely weak**, as for all big-M models: the relaxation sets $s_{ji} = 1/2$ and releases the precedences
- ▶ The integer optimum is $z(MILP) = 11$, sequence $2 \rightarrow 1 \rightarrow 3$: $\tilde{\kappa} = (9, 4, 15)$, $\tilde{\tau} = (6, 0, 5)$
- ▶ The given order cost 12; swapping the first two jobs saves 1

What the solver says (2/2)

UB (given order)	LB (hand dual)	$z(LP)$	$z(MILP)$	heuristic gap
12	2	2	11	9.1%



Additional considerations: transitivity (1/2)

- ▶ $s_{ji} \leq 1$ ($n(n-1)$) are valid and implied by constraints (1)
- ▶ **Transitivity:** for three distinct jobs j, i, k , if j precedes i and i precedes k then j precedes k :

$$s_{ji} \text{ AND } s_{ik} \implies s_{jk},$$

by De Morgan NOT s_{ji} OR NOT s_{ik} OR s_{jk} , that is

$$s_{ji} + s_{ik} - s_{jk} \leq 1$$

Additional considerations: transitivity (2/2)

- ▶ They are **valid** inequalities: in every real sequence transitivity holds, hence constraints (2) imply them (a cycle $j \rightarrow i \rightarrow k \rightarrow j$ would give $\kappa_j \geq \kappa_j + t_i + t_k + t_j$, impossible)
- ▶ In the relaxation they cut off the 1/2 solutions: they are the first step to strengthen a big-M model
- ▶ The choice of M : $\sum_j t_j$ is the smallest value that works in general; larger M leave the same integer set and a weaker relaxation

Exercise: minimising the maximum tardiness

- ▶ The company wants to minimise the tardiness of the latest job, not the sum
- ▶ Model and solve
- ▶ What changes in the link of τ_j ?

The final comparison (1/2)

The three numbers that every problem of the course produces:

	value	what it is
UB	12	heuristic solution
LB	2	certified bound
$z(MILP)$	11	MILP optimum

The final comparison (2/2)

- ▶ Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 9.1\%$$

- ▶ Certified gap: in absolute terms $UB - LB = 10$; as a percentage, against the heuristic solution,

$$\frac{UB - LB}{|UB|} = 83.3\%$$

- ▶ The heuristic is very good, the dual bound very poor: it is the signature of big-M models

A worked variant: release dates (1/2)

- ▶ Job 2 cannot start before time $\rho_2 = 2$ (the material arrives late); the others are available from the start
- ▶ A job starting no earlier than ρ_j completes no earlier than $\rho_j + t_j$: it suffices to strengthen the constraints to

$$\kappa_j \geq \rho_j + t_j, \quad \forall j \in \{1, 2, \dots, n\}$$

- ▶ **Mind the big-M:** the completions can now exceed $\sum_j t_j$ (the machine may stay idle waiting), and M must be updated to $\max_j \rho_j + \sum_j t_j$

A worked variant: release dates (2/2)

- ▶ On the instance the optimum becomes 12: the sequence $2 \rightarrow 1 \rightarrow 3$ would force the machine to wait until 2, with tardiness values $2 + 8 + 7$; the order $1 \rightarrow 2 \rightarrow 3$ stays at 12 and is optimal

	value	what it is
UB	12	heuristic solution
LB	2	hand-built dual certificate
$z(LP)$	4	relaxation without the bounds
$z(MILP)$	12	MILP optimum

Summary: Total tardiness

- ▶ Binary precedence variables s_{ji} , continuous completion variables κ_j and tardiness variables τ_j
- ▶ The disjunction “either j before i or i before j ” is linearised with the **big-M**, with $M = \sum_j t_j$
- ▶ $\tau_j \geq \kappa_j - d_j$ and $\tau_j \geq 0$: the link *is* the inequality; the equality with the maximum comes from optimality
- ▶ Bounds: $LB = 2$ (hand dual = $z(LP)$), $UB = 12$ (given order), $z(MILP) = 11$: the big-M relaxation is extremely weak

Summary

Final summary (1/3)

The common skeleton:

- ▶ jobs to be assigned to machines with limited availability
- ▶ what changes is what is paid and what is decided

The links encountered:

- ▶ **activation**, aggregated ($\sum_j t_{jm}x_{jm} \leq a_my_m$) and disaggregated ($x_j \leq y_c$)
- ▶ **maximum variable** ($y_m \geq t_{jm}x_{jm}$)
- ▶ **if and only if** (one direction from the constraint, the other from the objective)
- ▶ **big-M** and disjunctions ($M = \sum_j t_j$)

The general rule on links:

- ▶ one direction is always imposed by the **constraints**
- ▶ the other follows from the **objective** (or from the capacity constraint)
- ▶ if the datum is strictly positive one concludes “in every optimum”; if it may be zero, only “there exists an optimum”

Valid or optimality-preserving:

- ▶ a *valid* inequality cuts off no feasible integer solution
- ▶ an *optimality-preserving* constraint may cut some off, provided an optimum remains

Final summary (3/3)

The bounds of the seven problems:

Problem	LB	UB	$z(MILP)$	type
7.1 assignment	10	11	11	min
7.2 fixed cost	25/4	12	12	min
7.3 selection	20	34	25	max
7.4 parallel	5	19	15	min
7.5 classes with setup	9	100	21	max
7.6 completion bonus	32	150	42	max
7.7 tardiness	2	12	11	min

- ▶ The hand-built bounds are good where the relaxation is strong, poor where fixed costs and big-M appear